

Vulnerability Assessment Report

Target: OWASP Juice Shop Date:

May 10, 2026

Prepared for: Internship project

Table of Contents

1. Administrative Overview (Confidentiality & Disclaimer)
2. Assessment Overview & Scope
3. Finding Severity Ratings
4. Executive Summary
5. Attack Summary
6. Detailed Penetration Test Findings
7. Security Weakness / Conclusion
8. Evidence Gallery

1. Administrative Overview

Confidentiality Statement

This document contains proprietary and confidential information regarding the security posture of the target application. Duplication, redistribution, or use, in whole or in part, in any form, requires explicit consent from the authorized stakeholders.

Disclaimer

A vulnerability assessment is considered a snapshot in time. The findings and recommendations reflect the information gathered during the assessment and not any changes or modifications made outside of that period. Time-limited engagements do not allow for a full evaluation of all security controls. It is recommended to conduct similar assessments on a regular basis.

Contact Information

Assessment Type: External web application vulnerability assessment

Primary Evidence Sources: ZAP report, browser DevTools screenshots, and Nmap screenshot evidence

2. Assessment Overview & Scope

This report details the findings from an external vulnerability assessment conducted against the OWASP Juice Shop application. The assessment methodology blends automated scanning using ZAP 2.1 7.0 alongside manual verification and browser-level inspection.

Scope Note: Source material for this assessment indicates both a local testing instance (`http://localhost:3000`) captured via ZAP, and browser/DevTools evidence captured from the public staging instance (`preview.owasp-juice.shop`). Both represent the same OWASP Juice Shop application and its underlying architecture.

Assessment Components

- Automated DAST Scanning (ZAP 2.1 7.0)
- Manual Web Application Testing (Browser DevTools)
- Service Enumeration (Nmap)

Scope

- `http://localhost:3000` OWASP Juice Shop test instance reflected in ZAP findings
- `preview.owasp-juice.shop` browser evidence demonstrating frontend routes, sources, and network requests
- Observed routes and resources including `/rest/products/search` , `/rest/admin/application-configuration` , `/socket.io/` , `/sitemap.xml` , `/rest/continue-code` , `/#/score-board` , and `/#/jobs`

3. Finding Severity Ratings

The report utilizes the following standard risk severity model:

Severity	Score Range	Description
Critical	9.0 - 10.0	Exploitation is straightforward and usually results in system-level compromise or massive data loss.
High	7.0 - 8.9	Exploitation results in significant access to sensitive data or critical system functions.
Moderate	4.0 - 6.9	Vulnerabilities that require complex exploitation or result in limited, localized impact.
Low	0.1 - 3.9	Minor weaknesses, information disclosure, or defense-in-depth gaps.

4. Executive Summary

On May 10, 2026, a comprehensive security assessment was performed on the OWASP Juice Shop web application. The overall security posture of the application is weak, demonstrating critical flaws in database querying, cross-origin resource sharing, and client-side exposure.

The assessment confirmed **one high-risk SQL injection issue** that allows direct interaction with the backend database. Additionally, several medium-risk configuration weaknesses significantly increase the attack surface, notably permissive CORS policies, missing security headers, and session ID exposures.



5.Attack Summary

Finding	Severity	Evidence Source	Business Impact	Priority
1 . SQL Injection in Product Search	High	ZAP Report	Unauthorized database access, data exfiltration, loss of integrity.	Immediate
2. Overly Permissive CORS	Moderate	ZAP Report	Potential cross-site data theft and unauthorized API consumption.	High
3. Session ID Exposure in URL	Moderate	ZAP Report	Account takeover via session hijacking from referer logs.	High
4. Missing Anti-Clickjacking	Moderate	ZAP Report	Users tricked into unintended actions via malicious iframes.	Medium
5. CSP Header Not Set	Moderate	ZAP Report	Increased risk of successful Cross-Site Scripting (XSS) attacks.	Medium
6. Info Disclosure (Admin/Config)	Low	ZAP Report	Exposure of internal network topologies (IPs).	Medium
7. Client-Side Exposure & Route Enum	Low	Screenshots	Aids attackers in reverse-engineering application logic.	Medium
8. Additional Security Hardening Gaps	Low	ZAP / Screenshots	MIME sniffing, timestamp leakage, recruitment route exposure.	Medium

6. Detailed Penetration Test Findings

1.

SQL Injection in Product Search

High

Affected Asset: `http://localhost:3000/rest/products/search` (Parameter: `q`)

Risk Statement: The application fails to sanitize user input in the product search feature. This allows an attacker to manipulate backend database queries, potentially leading to unauthorized data extraction, modification, or complete database compromise.

Technical Description: ZAP identified that injecting specific SQL characters triggers an internal database error.

- ◆ **URL:** `GET http://localhost:3000/rest/products/search?q=%27%28`
- ◆ **Parameter:** `q`
- ◆ **Payload:** `'(`

Evidence: The server returned a HTTP 500 error leaking the raw SQLite query structure: `SELECT * FROM Products WHERE ((name LIKE '%(' OR description LIKE '%(') AND deletedAt IS NULL) ORDER BY name .`

Recommendation:-

1. Implement input validation and parameterized queries (Prepared Statements) for all database interactions.
2. Disable verbose error messages in production environments to prevent schema leakage.

2. Overly Permissive CORS / Cross-Domain Misconfiguration

Moderate

Affected Asset: `http://localhost:3000/assets/public/favicon_js.ico`

Risk Statement: The application utilizes an overly broad Cross-Origin Resource Sharing (CORS) policy. If applied to sensitive endpoints, this allows malicious third-party websites to read authenticated data from the victim's session.

Technical Description: The application responds with a wildcard CORS policy, signaling that any origin may read responses where browser protections otherwise apply.

- ◆
- ◆ **URL:** `http://localhost:3000/assets/public/favicon_js.ico`

Evidence: The response header explicitly contains `Access-Control-Allow-Origin: *`.

Recommendation:-

1. Restrict the `Access-Control-Allow-Origin` header to explicitly trusted domains only.

2. Avoid using the wildcard (*) operator on endpoints handling authentication or sensitive data.

3. Session Identifier Exposure in URL Rewrite Moderate

Affected Asset: `http://localhost:3000/socket.io/`

Risk Statement: Passing session identifiers in the URL query string exposes them to browser histories, proxy logs, and external sites via the Referer header, increasing the risk of session hijacking.

Technical Description: The application includes the live session token in the connection URL during Socket.IO communication, exposing it to logs and browser history.

- **URL:** `http://localhost:3000/socket.io/?EIO=4&transport=websocket&sid=pDlMUpWkc3tz-sqEAAAA`
- **Parameter:** `sid`

Recommendation:-

1. Transmit session identifiers strictly through secure, HttpOnly cookies or standard HTTP Authorization headers.

4. Missing Anti-Clickjacking Protection Moderate

Affected Asset: `POST / GET http://localhost:3000/socket.io/`

Risk Statement: The application does not prevent framing by external domains. Attackers can embed the application in an iframe on a malicious site, tricking users into performing unintended actions (Clickjacking).

Technical Description: ZAP noted the absence of both `X-Frame-Options` and CSP `frame-ancestors` directives on several endpoints.

- **Endpoint:** `GET / POST http://localhost:3000/socket.io/`
- **Evidence:** Anti-clickjacking protection was not present on the observed Socket.IO responses.

Recommendation:-

1. Implement the `X-Frame-Options: DENY` or `SAMEORIGIN` header.
2. Use Content-Security-Policy with the `frame-ancestors` directive.

5. Content Security Policy Header Not Set Moderate

Affected Asset: `http://localhost:3000/sitemap.xml` and general responses.

Risk Statement: Without a CSP, the browser has no strict guidelines on which dynamic resources are permitted to load, making the application heavily susceptible to Cross-Site Scripting (XSS) and data injection.

Technical Description: The `Content-Security-Policy` header is missing from HTTP responses across the application.

- ♦ **Endpoint:** `http://localhost:3000/sitemap.xml`
- ♦ **Evidence:** ZAP reported the CSP header as not set on the assessed response set.

Recommendation:-

1. Deploy a robust Content Security Policy limiting script execution to trusted origins.

6. Information Disclosure via Administrative Endpoints Low

Affected Asset: `http://localhost:3000/rest/admin/application-configuration`

Risk Statement: Administrative API endpoints leak internal infrastructure details, giving attackers valuable reconnaissance data.

Technical Description: Administrative and configuration endpoints returned internal environment details that are not required for normal users.

- ♦ **URL:** `http://localhost:3000/rest/admin/application-configuration`
- ♦ **Evidence:** ZAP detected a private IP disclosure (`192.168.99.100`) within the body of the application configuration response.

Recommendation:-

1. Ensure internal configuration endpoints require strict authentication and authorization.
2. Sanitize internal IP addresses and infrastructure details from API responses.

7. Client-Side Exposure and Route Enumeration Low

Affected Asset: Client-side JS chunks (`main.js` , `polyfills.js` , `scripts.js`) and `/#/score-board` .

Risk Statement: Publicly accessible source bundles reveal internal routing, challenge logic, and hidden features, heavily aiding reverse engineering.

Technical Description: Browser DevTools screenshots reveal that compiled JavaScript exposes internal logic and discoverable routes.

- ♦ **Evidence:** The `score-board` route and internal application menus are explicitly discoverable inside `main.js`.
- ♦ **Evidence:** Publicly accessible bundles including `main.js`, `polyfills.js`, and multiple chunk files are visible in the browser sources panel.

Recommendation:-

1. Minify and obfuscate production JavaScript to deter reverse engineering.
2. Do not rely on client-side routing obscurity to protect hidden endpoints.

8. Additional Security Hardening Gaps Low

Affected Asset: Multiple endpoints.

Risk Statement: Several defense-in-depth gaps exist that collectively weaken the application's overall security posture.

Technical Description: Multiple low-severity observations indicate inconsistent hardening and unnecessary disclosure of application metadata.

- ♦ Missing `X-Content-Type-Options` on `GET http://localhost:3000/socket.io/`.
- ♦ Unix timestamp disclosure (`1650485437`) on `http://localhost:3000/sitemap.xml`.
- ♦ Exposed application APIs via network traffic: `/api/Challenges/?name=Score%20Board`, `application-version`, `languages`, and `application-configuration`.
- ♦ Exposure of recruiting routes via custom headers (`X-Recruiting: /#/jobs`) in Nmap-observed response fingerprints.

Recommendation:-

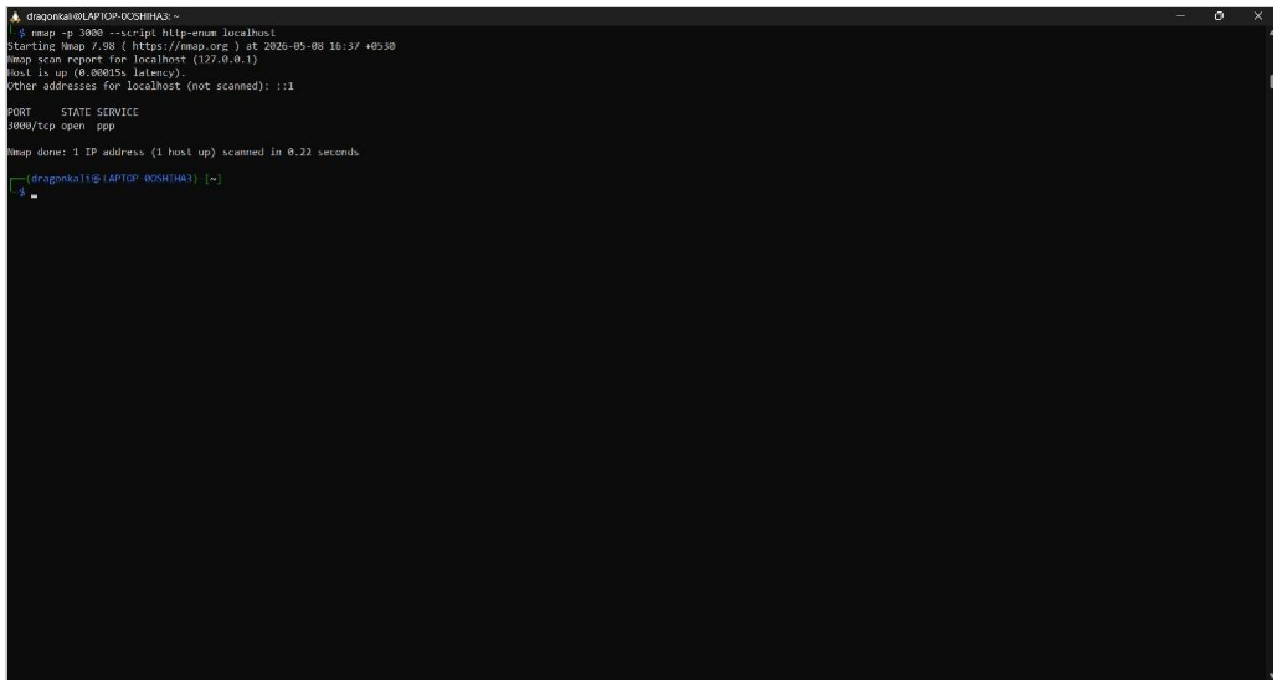
1. Enforce standard security headers across all endpoints.
2. Review and sanitize non-essential data (like timestamps or easter-egg headers) from HTTP responses.

7. Security Weakness / Conclusion

The OWASP Juice Shop application exhibits systemic security weaknesses spanning backend data validation, modern web application configuration, and client-server communication. The presence of a High-risk SQL Injection vulnerability demands immediate remediation, as it undermines the integrity and confidentiality of the entire data store. Furthermore, the numerous Moderate and Low risk findings demonstrate a lack of foundational security hardening (CORS, CSP, Anti-Clickjacking). Addressing these findings through standard secure coding practices and header configurations will vastly reduce the application's attack surface

8. Evidence Gallery

The following screenshots represent lab evidence collected during the internship project. They are included to support the narrative findings related to service discovery, application identification, and HTTP header analysis..

A terminal window with a dark background and light green text. The prompt is 'dragonkali@LAPTOP-00SH1H43: ~'. The command executed is 'nmap -p 3000 --script http-enum localhost'. The output shows the Nmap version (7.98), the scan target (localhost (127.0.0.1)), and the scan results. A table lists the open port 3000/tcp as 'http' with the service 'http'. The scan was completed in 0.22 seconds.

```
dragonkali@LAPTOP-00SH1H43: ~  
$ nmap -p 3000 --script http-enum localhost  
Starting Nmap 7.98 ( https://nmap.org ) at 2025-05-08 16:37 +0530  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.00015s latency).  
Other addresses for localhost (not scanned): ::1  
  
PORT      STATE SERVICE  
3000/tcp  open  http  
  
Nmap done: 1 IP address (1 host up) scanned in 0.22 seconds  
  
--(dragonkali@LAPTOP-00SH1H43) [~]
```

Figure 1. Nmap Port Enumeration Against localhost:3000

Confirms that TCP/3000 is open and reachable, supporting service exposure and scope validation for the local OWASP Juice Shop Instance

```
(dragonkali@LAPTOP-0C5H1H43:~)$ nmap -p 3000 -sV localhost
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-08 16:44 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00029s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
3000/tcp open  ppp?

Service detection performed. Please report any incorrect results at https://nmap.org/cgi-bin/submit.cgi?new-service :
Nmap done: 1 IP address (1 host up) scanned in 11.77 seconds
```

Figure 2. Nmap Service Fingerprint Identifying OWASP Juice Shop

Shows response fingerprinting that identifies the application as OWASP Juice Shop and exposes useful HTTP/header information for follow-on analysis

```
(dragonkali@LAPTOP-0C5H1H43:~)$ nmap -p 3000 -sV -st localhost
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-08 16:34 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00034s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
3000/tcp open  ppp?

Fingerprint-strings:
GET/200:200:
  HTTP/1.1 200 OK
  Access-Control-Allow-Origin: *
  X-Content-Type-Options: nosniff
  X-Frame-Options: SAMEORIGIN
  Feature-Policy: payment 'self'
  X-Recruiting: /#/jobs
  Accept-Ranges: bytes
  Cache-Control: public, max-age=0
  ETag: W/"126fa-19d8740487f"
  Content-Type: text/html; charset=UTF-8
  Content-Length: 75802
  Vary: Accept-Encoding
  Date: Fri, 08 May 2026 11:04:18 GMT
  Connection: close
  <!--
  Copyright (c) 2014-2026 Djero Kimlinch & the OWASP Juice Shop contributors.
  SPDX-License-Identifier: MIT
  </doctype html>
  <html lang="en" data-beasties-container>
  <head>
    <meta charset="utf-8">
    <title>OWASP Juice Shop</title>
    <meta name="description" content="probably the most modern and sophisticated insecure web application">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link id="favicon" rel="icon"
  HTTPOptions: HTTPRequest:
  HTTP/1.1 200 OK Content
  Access-Control-Allow-Origin: *
  Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
  Vary: Access-Control-Request-Headers
  Content-Length: 0
  Date: Fri, 08 May 2026 11:04:18 GMT
  Connection: close
  Help, MCP:
  HTTP/1.1 200 Bad Request
  Connection: close
Service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
```

Figure 3. Nmap HTTP Response / Header Evidence

Captures HTTP response characteristics used to support observations about permissive CORS and general security-header posture.

```
dragonkali@LAPTOP-OC5H1H43: ~  
Nmap done: 1 IP address (1 host up) scanned in 0.17 seconds  
  
--(dragonkali@LAPTOP-OC5H1H43) [~]  
$ nmap -p 3000 --script http-methods,http-title localhost  
Starting Nmap 7.98 ( https://nmap.org ) at 2025-05-08 16:49:40Z  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.00011s latency).  
Other addresses for localhost (not scanned): ::1  
  
PORT      STATE SERVICE  
3000/tcp  open  pop  
  
Nmap done: 1 IP address (1 host up) scanned in 0.15 seconds  
  
--(dragonkali@LAPTOP-OC5H1H43) [~]  
$
```

Figure 4. Nmap HTTP Title Probe
Demonstrates additional HTTP enumeration activity against port 3000 as part of service identification and validation.

```
dragonkali@LAPTOP-OC5H1H43: ~  
Nmap done: 1 IP address (1 host up) scanned in 11.77 seconds  
  
--(dragonkali@LAPTOP-OC5H1H43) [~]  
$ nmap -p 3000 --script http-headers localhost  
Starting Nmap 7.98 ( https://nmap.org ) at 2025-05-08 16:46:40Z  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.000098s latency).  
Other addresses for localhost (not scanned): ::1  
  
PORT      STATE SERVICE  
3000/tcp  open  pop  
  
Nmap done: 1 IP address (1 host up) scanned in 0.17 seconds  
  
--(dragonkali@LAPTOP-OC5H1H43) [~]  
$
```

Figure 5. Nmap HTTP Header Enumeration Attempt
Documents an additional header-enumeration step during reconnaissance, reinforcing that HTTP header inspection formed part of the internship assessment workflow.

